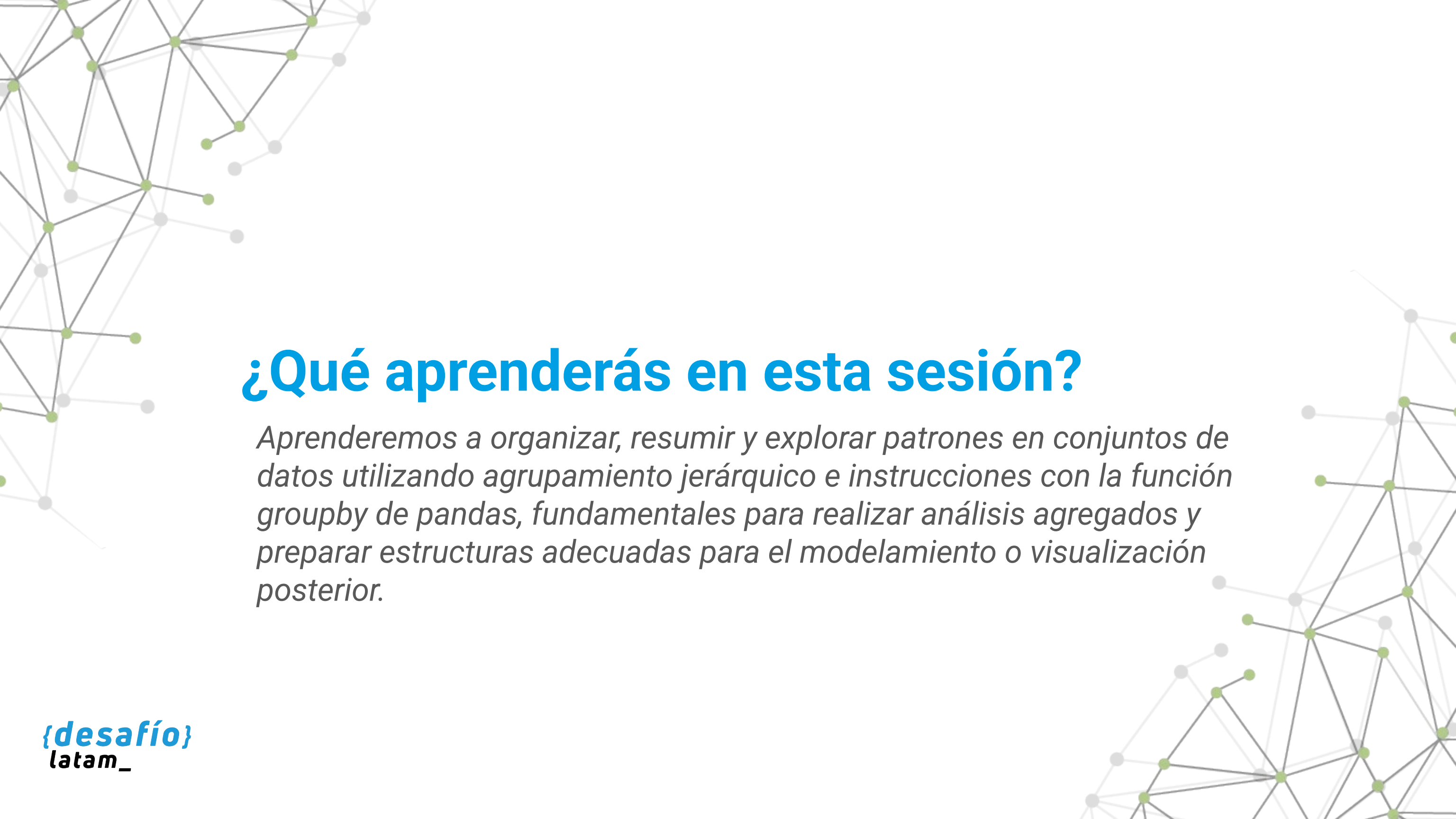




Limpieza y preparación de datos

Clase 5 - Agrupamiento y pivoteo de datos



¿Qué aprenderás en esta sesión?

Aprenderemos a organizar, resumir y explorar patrones en conjuntos de datos utilizando agrupamiento jerárquico e instrucciones con la función groupby de pandas, fundamentales para realizar análisis agregados y preparar estructuras adecuadas para el modelamiento o visualización posterior.

Desafío inicial

Estás trabajando con un conjunto de datos de ventas que contiene información sobre productos, fechas, ciudades y montos. La gerencia necesita reportes semanales que agrupen los datos por ciudad y semana, incluyendo estadísticas como total de ventas, promedio por producto y cantidad de transacciones.

¿Cómo podrías estructurar y agrupar esta información para responder rápidamente a estas consultas?

Para resolverlo, deberás conocer y aplicar las técnicas de agrupamiento jerárquico y la función groupby para agrupar, resumir y reordenar los datos de forma eficiente.

/* Agrupamiento y pivoteo de datos */

¿Qué es el agrupamiento en pandas?



Definición

El agrupamiento consiste en reorganizar un conjunto de datos para aplicar funciones de agregación sobre subconjuntos definidos por una o más columnas. Esto permite identificar patrones, realizar cálculos agregados y preparar datos para visualizaciones o modelos.



Implementación

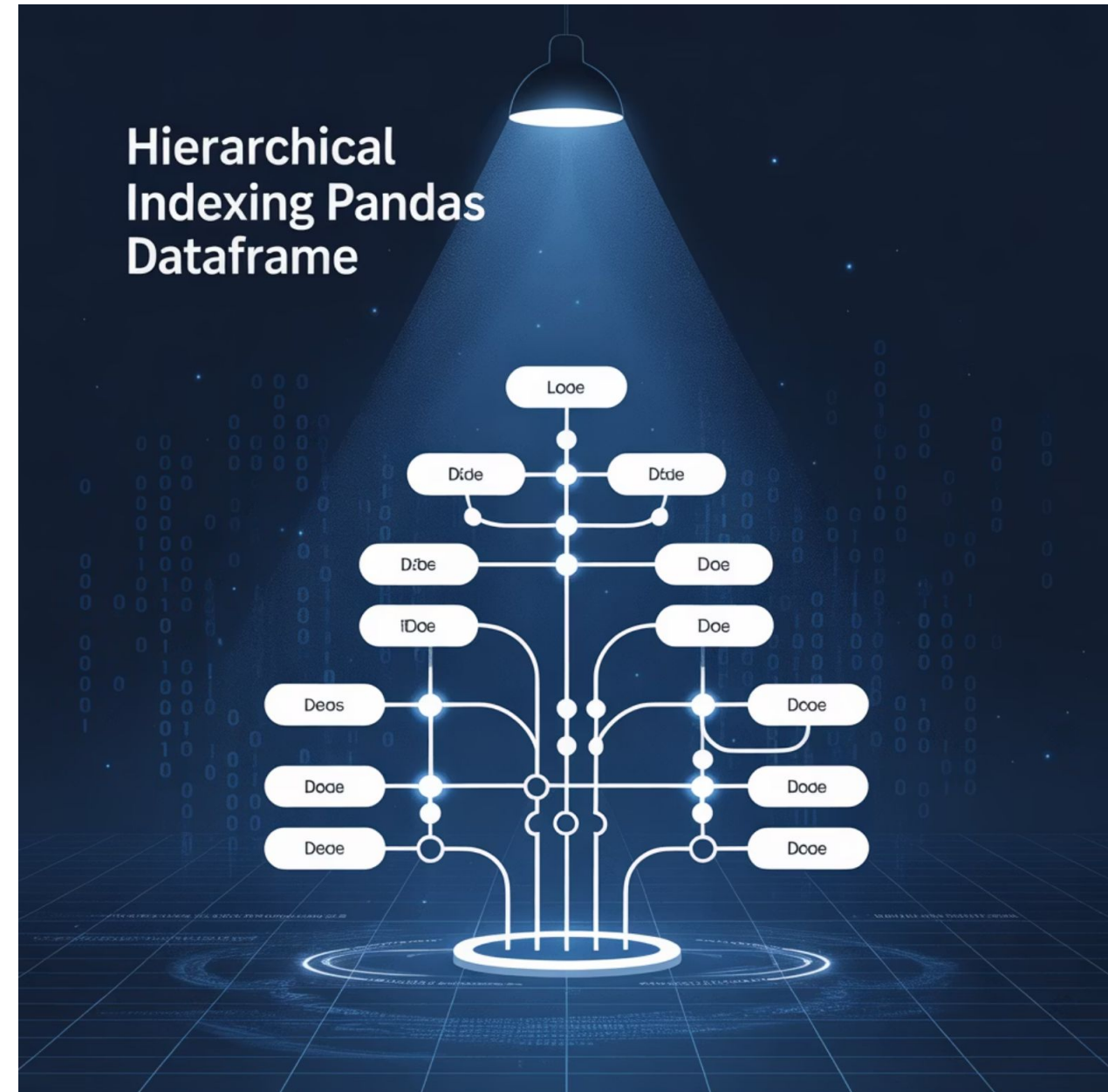
En pandas, esto se logra principalmente con la función `groupby`, pero también con técnicas como la indexación jerárquica.



Agrupamiento con indexación jerárquica

¿Qué es la indexación jerárquica y cómo facilita el análisis multidimensional de los datos?

La indexación jerárquica permite definir múltiples niveles de índice en un DataFrame, lo que es útil cuando queremos representar relaciones de agrupamiento anidadas. En pandas, esto



Ejemplo de indexación jerárquica

```
import pandas as pd
# Datos ficticios de ventas
data = {'Ciudad': ['Santiago', 'Santiago', 'Valparaíso', 'Valparaíso', 'Temuco'],
        'Semana': [1, 2, 1, 2, 1],
        'Ventas': [10000, 15000, 8000, 12000, 5000]}
df = pd.DataFrame(data)
df_grouped = df.groupby(['Ciudad', 'Semana']).sum()
print(df_grouped)
```

Ventajas de la indexación jerárquica

Este resultado tiene dos niveles de índice (Ciudad y Semana), permitiendo:

-  Acceder a los datos de forma anidada.
-  Realizar agregaciones jerárquicas.
-  Preparar los datos para un análisis o visualización estructurada.

Buenas prácticas

Usar `.sort_index()` para mantener el orden lógico de jerarquías.

Evitar niveles innecesarios que complejicen la lectura.

**/* Agrupamiento de datos con la función
groupby. */**

Agrupamiento con groupby

¿Cómo se aplica la función groupby para calcular estadísticas agrupadas?

La función groupby permite dividir el DataFrame según valores comunes de una columna (o varias) y aplicar funciones agregadas como sum(), mean(), count(), etc.

Ejemplo básico:

```
df.groupby('Ciudad')['Ventas'].sum()
```

Ejemplo avanzado:

```
df.groupby(['Ciudad',  
'Semana'])['Ventas'].agg(['sum',  
'mean', 'count'])
```

Esto genera una tabla agregada con múltiples estadísticas por grupo.

PANDAS GROUPBY AGGREGATION FUNCTIONS



Recomendaciones y comparación

Recomendaciones

Utilizar `as_index=False` si se desea mantener los datos en formato de tabla "plana".

Aplicar `.reset_index()` cuando se requiere aplanar el resultado para exportación o visualización.

¿Cuándo usar groupby y cuándo indexación jerárquica?

La elección depende de la finalidad del análisis:

- Si queremos mantener estructura jerárquica para acceder a subconjuntos: `MultiIndex` es útil.

- Si el objetivo es visualizar o exportar resultados simples: `groupby` con `reset_index()` es más directo.

Ambos enfoques se complementan y son clave para transformar datos brutos en resúmenes útiles.

Actividad guiada: Agrupando para decidir



Ejercicio

Agrupando para decidir

Objetivo de la actividad

Aplicar técnicas de agrupamiento jerárquico y groupby para generar estadísticas agregadas útiles en reportes de ventas.

Situación problema

Eres parte del equipo de análisis de una cadena de supermercados con sucursales en distintas ciudades. Necesitas responder rápidamente a las siguientes preguntas usando pandas:



¿Cuál fue el total de ventas por semana y ciudad?



¿Cuál fue el promedio de ventas semanales por ciudad?



¿Cuántas transacciones se realizaron por ciudad?



Ejercicio

Instrucciones

Paso 1: Cargar el archivo CSV

```
import pandas as pd
df =
pd.read_csv('ventas_semanales.csv')
```

Paso 2: Agrupar por ciudad y semana

```
resumen = df.groupby(['Ciudad',
'Semana'])['Ventas'].agg(['sum',
'mean', 'count'])
print(resumen)
```

Paso 3: Convertir en DataFrame "plano" para exportar

```
resumen.reset_index().to_csv('resumen_agrupado.csv', index=False)
```


Preguntas de reflexión y cierre

1

Insights

¿Qué insights relevantes puedes obtener a partir de este agrupamiento?

2

Decisiones

¿Qué decisiones se pueden tomar al ver esta información estructurada?

3

Automatización

¿Cómo podría integrarse este análisis en reportes automatizados?



Actividad práctica autónoma: Resumen de atención médica



Ejercicio

Resumen de atención médica

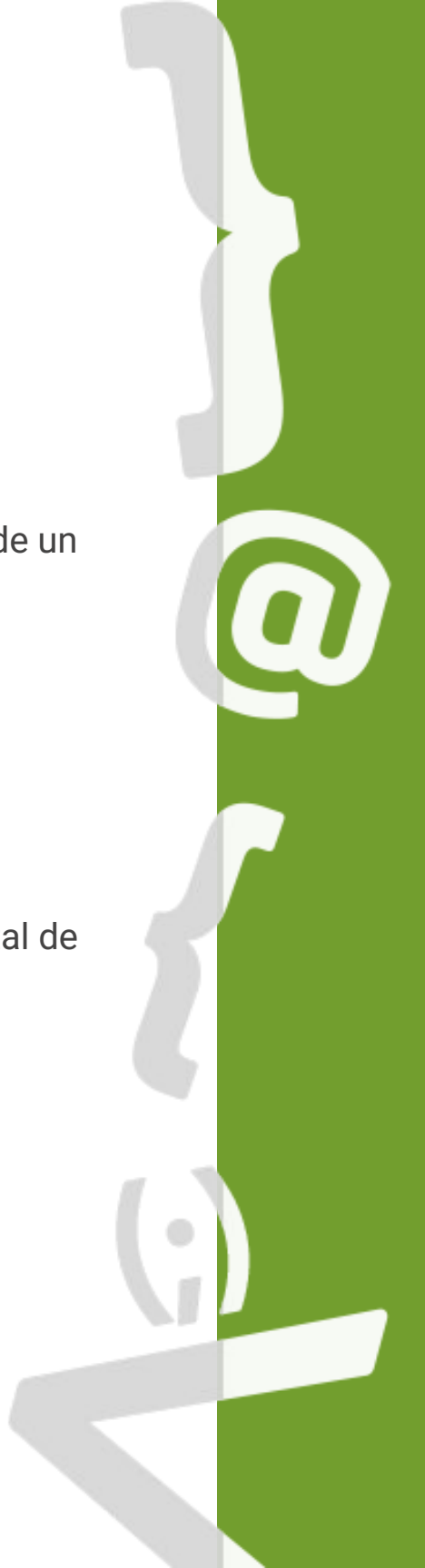
Objetivo de la actividad

Aplicar técnicas de agrupamiento con indexación jerárquica y la función groupby de la librería pandas para generar resúmenes estadísticos de un conjunto de datos sobre atenciones médicas.

Contexto

Una red de centros de salud privados requiere un resumen de atenciones médicas realizadas durante el primer trimestre del año.

La dirección médica necesita información agrupada por tipo de consulta (general o especializada) y por centro de atención, incluyendo el total de atenciones, promedio diario y cantidad de registros.



Ejercicio

Instrucciones

Copia y ejecuta el siguiente código para crear el DataFrame base:

```
import pandas as pd
datos = {'Centro': ['Clínica Norte', 'Clínica Norte', 'Clínica Sur', 'Clínica Sur',
'Clínica Centro', 'Clínica Centro', 'Clínica Norte', 'Clínica Sur'],
'TipoConsulta': ['General', 'Especialista', 'General', 'Especialista', 'General',
'Especialista', 'General', 'Especialista'],
'Atenciones': [120, 80, 100, 90, 110, 70, 130, 85]}
df = pd.DataFrame(datos)
```

Ejercicio

Instrucciones

Agrupar la información por 'Centro' y 'TipoConsulta', aplicando las siguientes funciones:

```
resumen = df.groupby(['Centro', 'TipoConsulta'])['Atenciones'].agg(['sum', 'mean', 'count'])  
print(resumen)
```

Ejercicio

Instrucciones

Exporta el resultado como archivo .csv usando:

```
resumen.reset_index().to_csv('resumen_atenciones.csv', index=False)
```

- Verifica la estructura final del archivo generado.

Reflexiona:

¿Por qué es útil agrupar por múltiples columnas en este caso?

¿Qué decisiones podrían tomarse a partir del promedio de atenciones por tipo de consulta?

¿Qué ventajas tiene exportar este tipo de resumen a CSV?



Resumen



Indexación jerárquica

Comprendimos qué es la indexación jerárquica y cómo se aplica para representar datos en múltiples niveles.



Función groupby

Utilizamos groupby para aplicar funciones agregadas sobre subconjuntos de datos.



Aplicación práctica

Aplicamos ambas técnicas en un ejercicio contextualizado, generando un resumen exportable.



Próxima sesión...

En la siguiente sesión aprenderemos a reorganizar y transformar la estructura de nuestros datos a través del pivoteo y despivoteo de DataFrames.

Veremos cómo trabajar con funciones como `pivot()` y `melt()` para lograrlo de manera eficiente.